BİL377 - HAFTA 07

Dinamik Programlama Temelleri

Tekrarlanan alt problemler, önbellek ve tablo kurma

$$W(n)=W(n-1)+W(n-2)$$

Merdivene çıkma yolları

Her adım 1 veya 2 basamak. Adımların sırası önemlidir.

Hedef: 4 basamağa ulaşan farklı adım dizilerinin sayısı

1+1+1+1 1+1+2 1+2+1

2+1+1 2+2

Toplam 5 farklı yol

1+2 ve 2+1 farklı yollardır.

Son adımın belirlediği iki grup

4. basamağa gelen bir yolun son adımı ya 1 ya da 2 basamaktır.

Son adım 1 ise önce 3. basamağa ulaşılmıştır

Son adım 2 ise önce 2. basamağa ulaşılmıştır

3. basamak için 3 yol, 2. basamak için 2 yol vardır

4. basamak için 3 + 2 = 5 yol

Gruplar ayrıdır ve bütün yolları kapsar.

Durum ve geçiş bağıntısı

W(i): i basamağa ulaşan adım dizilerinin sayısı.

$$W(0) = 1$$

$$W(1) = 1$$

$$W(i) = W(i-1) + W(i-2),\ i \geq 2$$

Aranan sonuç W(n)

W(0)=1, hiç adım atmayan boş diziyi sayar.

Merdiven tablosunun sayısal izi

Her satır kendinden önceki iki satırın sonucunu kullanır.

i	Hesap	W(i)
0	Boş adım dizisi	1
1	Yalnız 1	1
2	1 + 1	2
3	2 + 1	3
4	3 + 2	5
5	5 + 3	8

5 basamak için 8 yol vardır.

Aynı alt problemin tekrarı

Doğrudan özyinelemeli çözüm aynı W(i) değerini farklı dallarda hesaplar.

W(5) için W(4) ve W(3) gerekir

W(4) için W(3) ve W(2) gerekir

W(3) iki farklı istekten doğar

Saklanan W(3), ikinci isteği yeniden hesaplamadan yanıtlar

Tekrar eden soru aynıysa hesaplanan cevap yeniden kullanılabilir.

Böl yönet ve dinamik programlama

Alt problemlerin birbirleriyle ilişkisi çözüm yaklaşımını belirler.

Merge Sort: iki farklı alt dizinin sıralanması

Merdiven: aynı W(3) sorusunun farklı dallarda tekrarı

Memoization: durumu ilk istekte hesaplayıp saklama

Tabulation: durumları bağımlılık sırasıyla hesaplama

Sonuç saklamanın yararı, aynı durumun yeniden istenmesine bağlıdır.

Fibonacci sayıları

Fibonacci, aynı tekrar yapısını sade bir bağıntıyla gösterir.

$$F(0)=0,\ F(1)=1$$

$$F(n)=F(n-1)+F(n-2),\ n \geq 2$$

0, 1, 1, 2, 3, 5, 8, 13, …

Merdiven problemi için $W(n)=F(n+1)$

$F(5)=5$, fakat $W(5)=8$. Temel durumlar farklıdır.

Naif Fibonacci kodu


```
def fib_naive(n):
    dogrula(n)  # n negatif olmayan tam sayı
    if n < 2:   # F(0)=0 ve F(1)=1
        return n
    return fib_naive(n-1) + fib_naive(n-2)
```

Doğrudan bağıntı

fib_naive(5) = 5

Sol çağrı tamamen biter.

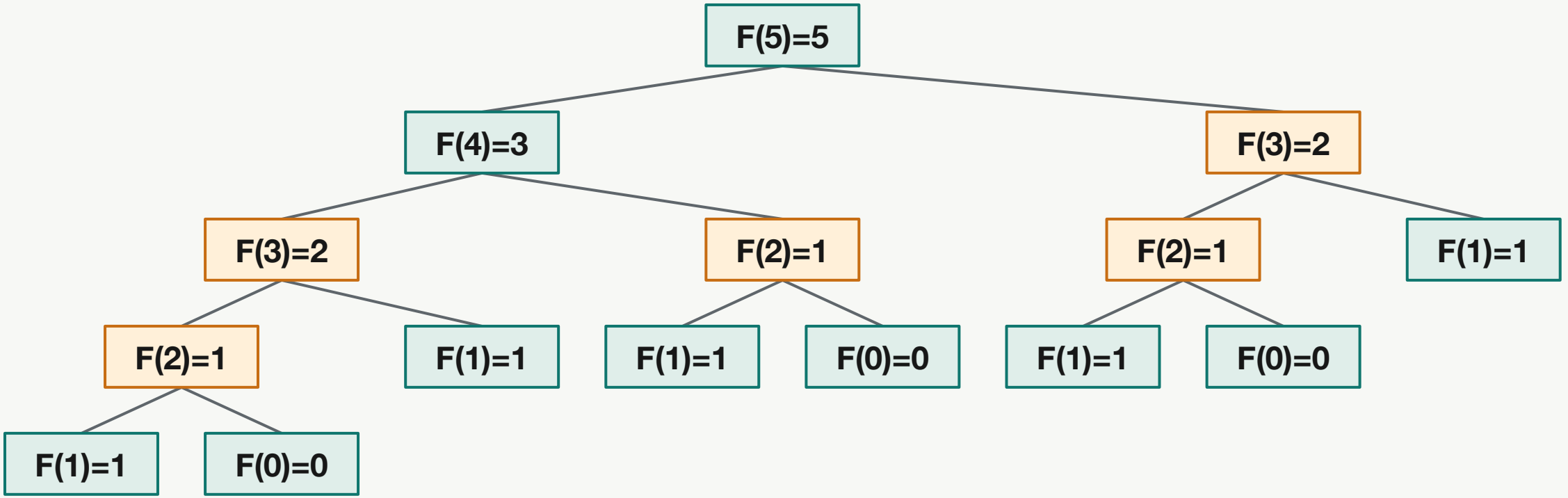
Sonra sağ çağrı çalışır.

İki sonuç toplanır.

Doğru sonuç, verimli hesap anlamına gelmez.

F(5) için özyineleme ağacı

Her düğüm bir çağrıdır. Düğüm etiketi: F(indeks)=dönen sonuç.



Turuncu: tekrar hesaplanan F(3) ve F(2) durumları

15 çağrı, 7 toplama, yalnızca 6 farklı durum

Ağaçta sonuçların yukarı dönmesi

Temel durumlar dönünce üst çağrı iki sonucu toplar.

Durum	Alt sonuçlar	Dönen değer	Bu hesap kaç kez?
F(2)	F(1)+F(0) = 1+0	1	3
F(3)	F(2)+F(1) = 1+1	2	2
F(4)	F(3)+F(2) = 2+1	3	1
F(5)	F(4)+F(3) = 3+2	5	1

Toplama sayısı: 3 + 2 + 1 + 1 = 7

F(5) hesabındaki tekrarlar

Her fonksiyona giriş, aynı parametre olsa da ayrı çağrı sayılır.

Parametre	Çağrı adedi	Dönen değer
5	1	5
4	1	3
3	2	2
2	3	1
1	5	1
0	3	0

Toplam 15 çağrı, 7 toplama. Farklı durum sayısı 6.

Naif çözümün maliyet bağıntısı

C(n): temel durumlar dahil toplam fonksiyon çağrısı.

$$C(0)=C(1)=1$$

$$C(n)=1+C(n-1)+C(n-2)$$

$$C(n)=2F(n+1)-1$$

$$\mathbf{C(5)=2\cdot 8-1=15}$$

Naif çözümün çağrı sayısı üstel büyür.

Çağrı yığınında aynı anda ne bekler?

F(5) ilk sol dala inerken, F(1) içindeki bir anın görüntüsü.

Yığındaki çağrı	Durumu
F(1) ← yığının üstü	1 değerini döndürmek üzere
F(2)	F(1) sonucunu ve ardından F(0) çağrısını bekler
F(3)	F(2) sonucunu ve ardından F(1) çağrısını bekler
F(4)	F(3) sonucunu ve ardından F(2) çağrısını bekler
F(5) ← ilk çağrı	F(4) sonucunu ve ardından F(3) çağrısını bekler

Toplam 15 çağrı yapılır, en fazla 5 Fibonacci çağrısı aynı anda bekler.

Naif çözümün zaman ve bellek büyümesi

Zaman bütün çağrıları, bellek ise en yüksek eşzamanlı ihtiyacı izler.

Çağrı sayısı $C(n)=2F(n+1)-1$

Zaman $\Theta(\phi^n)$, $\phi \approx 1{,}618$: üstel büyüme

$O(2^n)$ de geçerli, daha gevşek bir üst sınırdır

Yığın derinliği n: ek alan Θ(n)

Özyineleme ağacının genişlemesiyle yığının derinliği farklı büyür.

Memoization: hazır sonucu yeniden kullanma

Anahtar i, saklanan değer F(i). Her üst çağrı temiz bir sözlük açar.

İlk F(3) isteği sonucu hesaplar ve belleğe yazar

İkinci F(3) isteği bellekteki 2 değerini alır

Önbellek isabeti yeni bir alt çağrı ağacı açmaz

Özyineleme yapısı ve bağıntı aynı kalır

İstek sayısı ile gerçekten hesaplanan durum sayısı farklıdır.

Önbellekli Fibonacci kodu


```
def fib_memo(n):
    dogrula(n)
    bellek = {}  # Her üst çağrıda boş önbellek
    def hesapla(i):
        if i in bellek:  # Hazır mı?
            return bellek[i]
        if i < 2:
            sonuc = i
        else:
            sonuc = hesapla(i-1) + hesapla(i-2)
        bellek[i] = sonuc  # İlk sonucu sakla
        return sonuc
    return hesapla(n)
```

Yerel önbellek

bellek[i]: F(i).

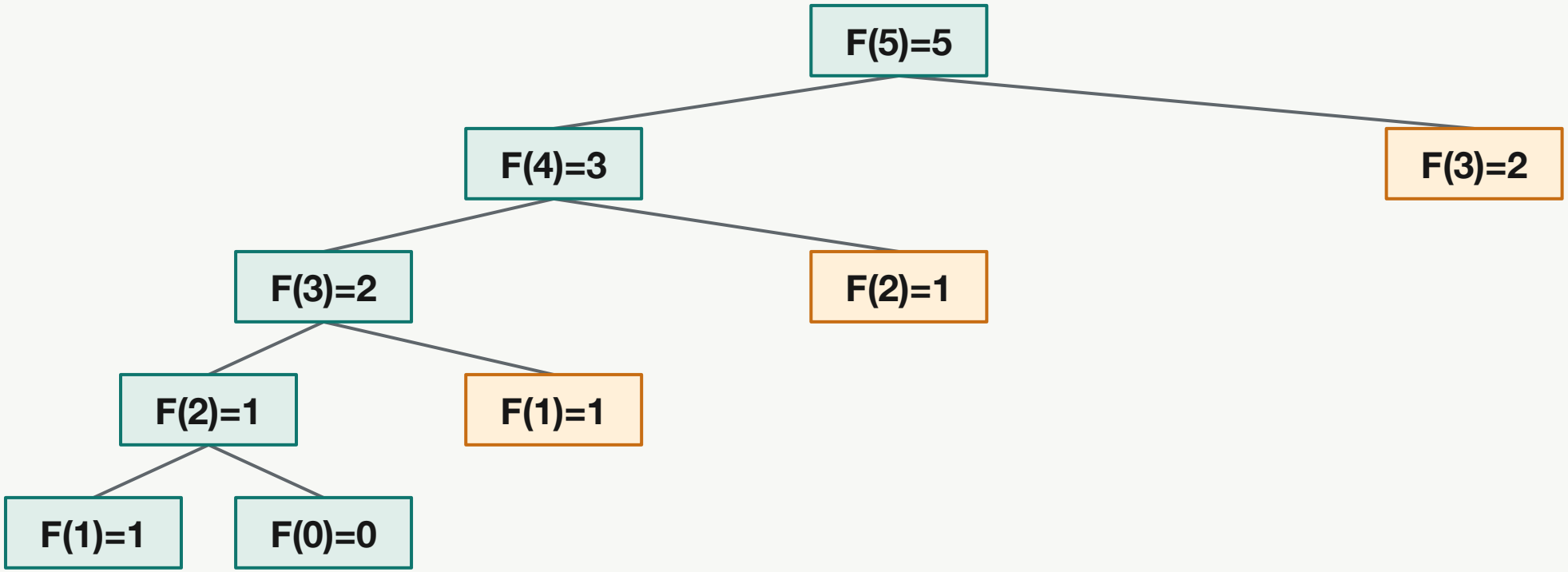
İsabet: hazır cevap.

İlk ziyaret: hesap ve kayıt.

Önbellek kontrolü, alt çağrılardan önce gelir.

Önbellekli F(5): hazır cevapta duran dallar

Turuncu düğüm hazır sonucu döndürür. Altına yeni çağrı açılmaz.



Turuncu: önbellek isabeti Yeşil: ilk kez çözülen durum

9 istek = 6 ilk hesap + 3 önbellek isabeti. Toplama sayısı 4.

Memoization için F(5) hesap sırası

Sol alt çağrı önce çalışır. İsabet, hazır sonucun okunmasıdır.

Olay	Tür	Sonuç
5, 4, 3, 2, 1	İlk iniş	F(1)=1 kaydı
0, sonra 2	Temel / hesap	F(0)=0, F(2)=1
1, sonra 3	İsabet / hesap	Hazır 1, F(3)=2
2, sonra 4	İsabet / hesap	Hazır 1, F(4)=3
3, sonra 5	İsabet / hesap	Hazır 2, F(5)=5

9 istek, 6 hesaplanan durum, 4 toplama.

Önbellekte hangi anahtar ve değerler var?

Anahtar Fibonacci indeksi, karşılığı o indeksin sonucudur.

Hesap tamamlanınca	Önbelleğin o andaki tamamı
F(2)=1	{0:0, 1:1, 2:1}
F(3)=2	{0:0, 1:1, 2:1, 3:2}
F(4)=3	{0:0, 1:1, 2:1, 3:2, 4:3}
F(5)=5	{0:0, 1:1, 2:1, 3:2, 4:3, 5:5}

İkinci F(3) isteği, 3 anahtarında saklanan 2 sonucunu okur.

Memoization zaman ve bellek hesabı

F(0)..F(n) için n+1 farklı durum vardır, n ≥ 2.

Her yeni durumda en fazla iki alt sonuç kullanılır

Sözlük erişimi beklenen sabit maliyetli kabul edilir

Toplam beklenen zaman Θ(n)

Önbellek Θ(n), çağrı yığını Θ(n)

Sonuçları saklamak tekrar hesabını azaltır, çağrı yığını kalır.

Durum bağımlılıkları ve hesap sırası

F(i), yalnızca daha küçük iki indekse bağlıdır.

F(0) ve F(1) başlangıçta bilinir

F(2) için 0 ve 1 hazırdır

F(3) için 1 ve 2 hazırdır

Sıra: 0, 1, 2, 3, …, n

Bu örnekte bağımlılıklar döngü içermez.

Tabulation: aşağıdan yukarıya Fibonacci


```
def fib_tablo(n):
    dogrula(n)
    if n < 2:
        return n
    dp = [0] * (n+1)  # 0..n için yer
    dp[1] = 1         # F(0)=0, F(1)=1
    for i in range(2, n+1):  # n dahil
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

dp[i] = F(i)

n+1 hücre.

Başlangıç: dp[0]=0, dp[1]=1.

i=2..n sırasında hesap.

Kodda n+1 üst sınırı, n indeksini döngüye dahil eder.

F(5) tablosunun adım adım kurulması

dp[i], F(i) sonucunu saklar. Başlangıçta 6 hücre ayrılır.

Adım	Hesap	dp dizisinin tamamı
Başlangıç	F(0)=0, F(1)=1	[0, 1, 0, 0, 0, 0]
i=2	1 + 0 = 1	[0, 1, 1, 0, 0, 0]
i=3	1 + 1 = 2	[0, 1, 1, 2, 0, 0]
i=4	2 + 1 = 3	[0, 1, 1, 2, 3, 0]
i=5	3 + 2 = 5	[0, 1, 1, 2, 3, 5]

6 hücre, 4 toplama, sonuç dp[5]=5

Tablonun doğruluk gerekçesi

Döngü değişmezi: i hesabından önce dp[0..i−1] doğrudur.

Başlangıç: dp[0]=0 ve dp[1]=1 doğrudur

Adım: doğru iki önceki değer toplanır

Yeni dp[i], Fibonacci bağıntısıyla F(i) olur

Bitiş: i=n tamamlandığında dp[n]=F(n)

Doğru bağıntı, doğru temel durum ve doğru hesap sırası birlikte gerekir.

Tabulation zaman ve bellek maliyeti

$n \geq 2$ için döngü tam $n-1$ kez çalışır.

Dizi oluşturma: $\Theta(n)$

Hücre güncellemeleri: $n-1$ toplama

Toplam zaman $\Theta(n)$, tablo alanı $\Theta(n)$

Özyineleme yığını bulunmaz

Doğrusal tablo alanı ile sabit çağrı yığını ayrı büyüklüklerdir.

Bütün tabloya ihtiyaç var mı?

F(i) hesabı yalnız F(i–1) ve F(i–2) değerlerini okur.

F(5) için F(4)=3 ve F(3)=2 yeterlidir

F(0)..F(2), F(5) hesabında artık okunmaz

İki önceki sonuç iki değişkende saklanabilir

Tablo boyutu Θ(n), tutulan sayı adedi Θ(1) olur

Bellek azaltma, artık okunmayacak durumların bırakılmasıdır.

İki değişkenli Fibonacci


```
def fib_iki(n):
    dogrula(n)
    if n < 2:
        return n
    onceki, simdiki = 0, 1  # F(0), F(1)
    for i in range(2, n+1):
        onceki, simdiki = simdiki, onceki + simdiki
    return simdiki  # Son güncellemede F(n)
```

Eski değerler birlikte okunur

onceki = F(i–2)

simdiki = F(i–1)

Atamadan sonra simdiki = F(i).

Python çoklu atamada sağ tarafı önce değerlendirir.

F(5) için iki değişkenin sayısal izi

Başlangıç çifti (0,1). Her satır eski çiftin değerlerini okur.

i	Eski çift	Yeni çift
2	(0,1)	(1,1)
3	(1,1)	(1,2)
4	(1,2)	(2,3)
5	(2,3)	(3,5)

4 toplama, 2 kalıcı Fibonacci değişkeni, dönen sonuç 5

Dört yöntemin karşılaştırılması

Aynı F(n) sonucu, farklı tekrar ve saklama maliyetleri.

Yöntem	Zaman	Saklanan sonuçlar	Çağrı yığını
Naif	$\Theta(\phi^n)$	Önbellek yok	$\Theta(n)$
Önbellekli DP	Beklenen $\Theta(n)$	$\Theta(n)$ sözlük	$\Theta(n)$
Tablolu DP	$\Theta(n)$	$\Theta(n)$ tablo	$\Theta(1)$
İki değişkenli DP	$\Theta(n)$	$\Theta(1)$ sayı adedi	$\Theta(1)$

Ek alan: naif $\Theta(n)$, memo $\Theta(n)$, tablo $\Theta(n)$, iki değişken $\Theta(1)$

F(5) için sayılarla zaman ve bellek

Sonuç her yöntemde 5. Yığın yalnız hesap çağrılarını sayar.

Yöntem	Çağrı / istek	Toplama	Saklanan sonuçlar	En derin yığın
Naif	15	7	Önbellek yok	5 çağrı
Önbellekli DP	9	4	6 sözlük kaydı	5 çağrı
Tablolu DP	1	4	6 hücre	1 çağrı
İki değişkenli DP	1	4	2 kalıcı değişken	1 çağrı

Saklanan sonuç ve yığın sütunları ayrı tepe adetleridir.

F(25) için ölçülen süre ve tahsisler

F(25)=75025. Sayaçsız kod. CPython 3.14.6, macOS / arm64.

Yöntem	Toplama	Ortanca süre (ms)	İzlenen tepe (bayt)
Naif	121392	11.989875	288
Önbellekli DP	24	0.002958	2304
Tablolu DP	24	0.000667	856
İki değişkenli DP	24	0.000416	248

Süre ve bellek ayrı deneylerdir. İzlenen tahsis, toplam RAM değildir.

Bellek adedi, bayt ve Python tam sayıları

Algoritmik alan hesabı ile araçla ölçülen tahsis farklı şeyleri anlatır.

Θ(n) alan: n ile büyüyen hücre veya çağrı sayısı

Θ(1) sayı adedi: sabit sayıda Fibonacci değişkeni

Python int büyüdükçe sayının bayt ihtiyacı da artar

tracemalloc: izlenen tahsisler, tam RAM / yığın değil

İki değişken kullanmak, her büyüklükte n için sabit bayt garantisi vermez.

İşlem sayısı ve süre aynı şey değildir

Sayaçlı deney: çağrı, durum ve toplama sayılarını süreyle birlikte raporlar.

Naif: her giriş ve her toplama sayılır

Memo: istek, yeni hesap ve isabet ayrıdır

Tablo: hücre sayısı ve n–1 toplama raporlanır

Süre: sayaçlı çağrının 3 tekrarının ortancası

Süreye çıktı yazdırma dahil değildir. Sayaçların yükü dahildir.

Fibonacci için kesin sayımlar

n=5 ve n=10, kodun sayımlarıyla karşılaştırılabilir küçük örnekler.

n / yöntem	İstek	Yeni hesap	Toplama
5 / naif	15	15	7
5 / memo	9	6	4
10 / naif	177	177	88
10 / memo	19	11	9

Tablo ve iki değişkenli sürüm, n=10 için 9 toplama yapar.

Önbellek ölçümünde başlangıç durumu

Hazır önbellekle tekrar çağrı, baştan hesaplamadan farklı bir deneydir.

İlk fib_memo(25): alt durumlar hesaplanır

Yerel sözlük çağrı sonunda bağımsız kalır

İkinci fib_memo(25): yeni boş sözlük açılır

Her ölçüm aynı başlangıç koşuluyla karşılaştırılır

Sıcak önbellekten tek okuma, tüm hesabın süresi değildir.

Merdiven probleminin tablo kodu


```
def merdiven(n):
    dogrula(n)
    dp = [0] * (n+1)
    dp[0] = 1
    for i in range(1, n+1):
        dp[i] = dp[i-1]
        if i >= 2:
            dp[i] += dp[i-2]
    return dp[n]
```

dp[i] yol sayısıdır

dp[0]=1.

Son adım 1: dp[i−1].

Son adım 2: dp[i−2].

n=5 için sonuç 8. Hiçbir yol dizisi tek tek üretilmez.

Dinamik programlama modelinin parçaları

Model, koddan önce durumun anlamını açıklar.

Durum: dp[i] hangi sorunun cevabı?

Başlangıç: hangi cevaplar doğrudan biliniyor?

Geçiş: önceki cevaplardan yeni cevap nasıl çıkıyor?

Sıra ve çıktı: hangi hücreler ne zaman, sonuç nerede?

Merdivende durum yol sayısıdır. Fibonacci'de durum dizi değeridir.

Minimum maliyetli aşamalar

Bir işlem 1 veya 2 aşama ilerler. Uğradığı aşamanın ücretini öder.

Başlangıç 0 ücretsiz, hedef 4. aşama

1, 2, 3, 4. aşama ücretleri: 10, 15, 20, 5

0–1–3–4 maliyeti: 10+20+5=35

0–2–4 maliyeti: 15+5=20

Hedef aşamanın ücreti de toplamın içindedir.

Minimum maliyet bağıntısı

M(i): başlangıçtan i. aşamaya ulaşmanın en az toplam ücreti.

$M(0)=0,\ M(1)=10$

$M(i)=\text{ücret}[i-1]+\min(M(i-1), M(i-2)),\ i \geq 2$

$M(2)=15+\min(10,0)=15$

$M(3)=20+\min(15,10)=30$

Ücret dizisi sıfır tabanlıdır. i. aşamanın ücreti ücret[i–1].

Minimum maliyet tablosu

Ücretler [10,15,20,5]. Eşitlikte önceki aşama seçilir.

Aşama	En iyi geçmiş	Maliyet	Önceki
0	Başlangıç	0	—
1	M(0)=0	10	0
2	min(10,0)=0	15	0
3	min(15,10)=10	30	1
4	min(30,15)=15	20	2

En az tutar 20. Geri izleme: 4, 2, 0.

Maliyet hesabı ve seçim kaydı


```
dp = [0] * (n+1)
ata = [0] * (n+1)
for i in range(1, n+1):
    j = i-1
    if i >= 2 and dp[i-2] < dp[j]:
        j = i-2
    dp[i] = dp[j] + ucretler[i-1]
    ata[i] = j
yol, i = [], n
while i > 0:
    yol.append(i)
    i = ata[i]
yol.reverse()
```

dp ve ata birlikte

Tam fonksiyon Python dosyasında.

dp: en az tutar.

ata: seçilen önceki aşama.

Sonuç: 20, [2,4].

Dönen sonuç: (20, [2,4]). Başlangıç 0 listeye dahil değildir.

En iyi alt çözümün kullanılma gerekçesi

Bir aşamadan sonrası yalnız o aşamaya ve ilerideki ücretlere bağlıdır.

i. aşamaya son geliş i–1 veya i–2 üzerinden olur

Aynı önceki aşamaya daha ucuz bir yol varsa toplam düşer

Bu nedenle önceki aşamanın en iyi maliyeti yeterlidir

Geçmişe bağlı indirim varsa durum daha fazla bilgi ister

Durum, gelecekteki kararları etkileyen bilgiyi korumalıdır.

Uygulamadaki karşılığı

Aynı kalan alt hesapları tekrar üretmek yerine hazır sonuç kullanılır.

Aşamalı süreçlerde en ucuz geçerli karar dizisi

Ölçümlerde aynı ara hesabın yeniden kullanılması

İleride: para üstü, kapasite ve metin hizalama

Kazanç: daha az tekrar işlem ve daha kısa bekleme

Her problem için durum ve bağıntı ayrı tasarlanır.

Sınır durumları ve hata sözleşmesi

Yanlış tür ve negatif indeks geçerli problem örneği değildir.

Girdi	Beklenen sonuç	Gerekçe
F(0), F(1)	0, 1	Temel durum
W(0), W(1)	1, 1	Boş yol ve tek adım
n=−1	ValueError	Negatif indeks yok
n=2.5 veya True	TypeError	Tam sayı sözleşmesi
Ücretler []	(0, [])	Başlangıç zaten hedef

Memo sürümü büyük n değerinde Python çağrı sınırına ulaşabilir.

Alıştırmalar: durum, değer ve maliyet

Merdivende adımlar 1 veya 2 basamak. Fibonacci için F(0)=0, F(1)=1.

6 basamak için W(6) kaçtır?

F(6) hesabında naif çağrı ve memo toplama sayıları?

[10,15,20,5] için en ucuz yol hangi aşamalardan geçer?

İki ayrı atama, Fibonacci güncellemesini neden bozabilir?

Her soruda durumun anlamı ve kullanılan kod sürümü önemlidir.

Alıştırmalar: yeni merdiven kuralı

Her adım artık 1, 2 veya 3 basamak olabilir. Adım sırası önemlidir.

A(0), A(1) ve A(2) başlangıçları nedir?

A(n) için hangi üç alt durum gerekir?

A(3) ve A(4) kaç farklı yol sayar?

Zaman ve tablo alanı nasıl büyür?

Bu değişiklik, ödevdeki modelin temelidir.

Ödev: 1, 2 veya 3 basamaklı yollar

Aynı problem için naif, memoization ve tabulation çözümleri.

Sözleşme: $n \geq 0$, $A(0)=1$, adım sırası önemli

Kontrol değerleri: 1, 1, 2, 4, 7, 13

Rapor: çağrı, toplama, süre ve bellek modeli

Teslim: merdiven_dp.py, rapor.md, sonuclar.csv

Tam yönerge ve ölçüm koşulları ODEV.md dosyasındadır.

Ders örneklerinin program arayüzü

Kod klasöründeki programlar: ağaç, önbellek, zaman/bellek ve uygulama.

```
fibonacci_incele.py --ornek agac --n 5

fibonacci_incele.py --ornek memo --n 5

fibonacci_incele.py --ornek karsilastirma

ders_ornekleri.py --ornek uygulama
```

Tam komutlar ve örnek çıktılar hafta klasöründeki README.md dosyasındadır.

Değerlendirme soruları

Aynı bağıntının farklı yürütme biçimleri farklı kaynaklar kullanır.

Memoization hangi tekrar hesabını ortadan kaldırır?

Tabulation için hesap sırası nasıl belirlenir?

Sabit sayıda değişken tutmak ne zaman yeterlidir?

Bir sonraki hafta: para üstü ve kapasite durumları

Durum anlamı, geçiş, temel durum ve çıktı birlikte çözümü tanımlar.